🧠 Exercício Prático – Sistema de Quiz em C# (Console)

🎯 Enunciado

Desenvolva um **sistema de quiz (jogo de perguntas)** utilizando **C# no console**, com o objetivo de reforçar conceitos fundamentais da linguagem e boas práticas de programação.

O sistema deverá apresentar perguntas ao usuário, permitir a escolha de respostas, validar os resultados e exibir a pontuação final, além de possibilitar a repetição do jogo.

📚 Objetivos de Aprendizagem

Ao final deste exercício, o aluno deverá ser capaz de:

- Aplicar **classes e encapsulamento**
- Utilizar **enumerações (enum)**
- Trabalhar com **listas genéricas (`List<T>`)**
- Utilizar **LINQ (Where / OrderByDescending)**
- Aplicar **expressões lambda**
- Controlar fluxo com **while, foreach**
- Utilizar **if e switch**
- Implementar **tratamento de exceções**
- Criar e utilizar uma **classe estática utilitária**

🏗️ Estrutura do Sistema (Modelagem)

📦 Classe: `Pergunta`

Responsabilidade: Representar uma pergunta do quiz.

🔹 Atributos

- `Enunciado` : string
- `Alternativas` : List
- `RespostaCorreta` : Alternativa

🔹 Métodos

- Construtor para inicializar os dados da pergunta

📦 Enum: `Alternativa`

Responsabilidade: Representar as opções de resposta.

🔹 Valores

- A
- B
- C
- D

📦 Classe: `Quiz`

Responsabilidade: Controlar a execução do jogo.

🔹 Atributos

- `Perguntas` : List
- `Pontuacao` : int

🔹 Métodos

- `Iniciar()`
- `ExibirPergunta(pergunta)`
- `LerResposta()`
- `ValidarResposta(pergunta, resposta)`
- `ExibirResultado()`

📦 Classe Estática: `QuizUtils`

Responsabilidade: Métodos auxiliares reutilizáveis.

🔹 Métodos

- `ExibirMensagem(texto)`
- `EmbaralharPerguntas(listaPerguntas)`

📦 Classe: `Jogador` (Opcional)

Responsabilidade: Representar um jogador no ranking.

🔹 Atributos

- `Nome` : string
- `Pontuacao` : int

🔗 Relacionamento entre Classes

Quiz
├── List<Pergunta>
│ ├── Enunciado
│ ├── Alternativas (List<string>)
│ └── RespostaCorreta (Alternativa)
│
├── Pontuacao
│
└── Métodos de execução do jogo

QuizUtils (static)
└── Métodos auxiliares

Jogador (opcional)
├── Nome
└── Pontuacao

🔄 Fluxo de Execução

▶️ Fluxo Geral

[INÍCIO]
↓
Criar Quiz
↓
Carregar Perguntas
↓
(while)
↓
Iniciar Quiz
↓
[Para cada pergunta]
↓
Exibir Pergunta
↓
Ler Resposta (try/catch)
↓
Validar (if)
↓
Atualizar Pontuação
↓
Exibir Resultado
↓
Perguntar se deseja repetir
↓
FIM

🧠 Fluxo Interno do Quiz

Iniciar()
↓
Zerar pontuação
↓
foreach perguntas
↓

ExibirPergunta()
  ↓
LerResposta()
  ↓
ValidarResposta()
  ↓
Somar pontos
  ↓
ExibirResultado()

🧪 Lista Inicial de Perguntas

Implemente obrigatoriamente as perguntas abaixo:

❓ Pergunta 1

Qual estrutura permite armazenar uma coleção tipada em C#?

A) ArrayList
B) List
C) Hashtable
D) Stack não genérica

Resposta correta: B

❓ Pergunta 2

Qual palavra-chave é usada para tratar exceções?

A) try
B) catch
C) throw
D) Todas as alternativas

Resposta correta: D

❓ Pergunta 3

O que é um enum em C#?

A) Uma classe abstrata
B) Um tipo que define constantes nomeadas
C) Um método estático
D) Um tipo dinâmico

Resposta correta: B

❓ Pergunta 4

Qual método do LINQ ordena do maior para o menor?

A) OrderBy
B) Sort
C) OrderByDescending
D) Reverse

Resposta correta: C

❓ Pergunta 5

Qual estrutura é usada para repetir um bloco enquanto uma condição é verdadeira?

A) if
B) switch
C) while
D) break

Resposta correta: C

▶️ Passo a Passo Guiado

🔹 Passo 1 – Criar Estruturas

- Criar `Pergunta`, `Quiz`, `Alternativa` e `QuizUtils`

🔹 Passo 2 – Criar Lista de Perguntas

- Utilizar `List<Pergunta>`
- Adicionar perguntas manualmente

🔹 Passo 3 – Exibir Perguntas

- Usar `foreach`
- Mostrar enunciado e alternativas

🔹 Passo 4 – Ler Resposta

- Usar `Console.ReadLine()`
- Converter para enum
- Aplicar `try/catch`

🔹 Passo 5 – Validar

- Usar `if` ou `switch`
- Comparar com resposta correta

🔹 Passo 6 – Loop do Jogo

- Usar `while`
- Permitir repetir

🔹 Passo 7 – Resultado

- Mostrar pontuação final

🔹 **Passo 8 – LINQ**

- Aplicar `Where` ou `OrderByDescending`

🚀 Desafios Extras

🧩 1. Tempo por Pergunta

Implemente um tempo limite (ex: 10 segundos).

💡 Dicas:

- Utilizar `DateTime` ou `Stopwatch`
- Namespace: `System.Diagnostics`

✔ Regra:

- Se o tempo for excedido → resposta inválida

🧩 2. Embaralhar Perguntas

💡 Dica:

- Utilizar `Random` ou LINQ

🧩 3. Nível de Dificuldade

Criar enum:

- Fácil
- Médio
- Difícil

🧩 4. Ranking de Jogadores

Criar classe `Jogador` e ordenar com:

- `OrderByDescending`

🧠 Desafio Final

➕ Permitir adicionar novas perguntas

O sistema deve permitir:

- Inserir novas perguntas via console
- Adicionar na `List<Pergunta>`

💡 Sugestão:

Criar método:

- `AdicionarPergunta()`

🏁 Entrega

O aluno deverá entregar:

- Código fonte completo
- Projeto funcional no console
- Implementação dos requisitos obrigatórios
- (Opcional) Desafios extras

💬 Considerações Finais

Este exercício foi projetado para simular um sistema real, integrando diversos conceitos fundamentais de C#.

O foco está em **organização, clareza e aplicação correta dos conceitos**, não apenas no funcionamento do programa.

🚀 Priorize código limpo, bem estruturado e de fácil entendimento.